RAG Fundamentals & LLM Accuracy

Complete Session Script

Duration: 20-25 minutes

Topics Covered:

Improving LLM Accuracy & Results

Prerequisites for RAG Implementation

What is RAG & Complete Workflow

Opening

Duration: 1 minute

Good morning/afternoon everyone. Today we're going to talk about something that's become critical in building production LLM applications - how to make these models actually useful and accurate for your specific needs. We'll cover how to improve LLM accuracy and results, what you need before implementing RAG, and then do a deep dive into what RAG actually is and how to build it.

Let me start with a problem we've all encountered. You use ChatGPT or any LLM and ask it about your company's latest product features or internal policies, and it either makes up plausible-sounding nonsense or tells you 'I don't have information about that.' The model is incredibly intelligent but knows nothing about your specific data. By the end of our session today, you'll understand exactly how to solve this problem. Let's dive in.

Section 1: How to Improve LLM Accuracy and Results

Duration: 6 minutes

Before we even get to RAG, let's talk about the foundational ways to improve LLM outputs. Understanding these approaches will help you appreciate why RAG is so powerful and when to use it versus other techniques.

There are five main strategies for improving LLM accuracy, and they exist on a spectrum from simple to complex, from cheap to expensive.

Strategy One: Prompt Engineering

This is your first line of defense and often the most cost-effective. Good prompt engineering means being specific, providing context, using examples, and structuring your prompts clearly.

Instead of asking 'Write a product description,' you'd say: **Write a 150-word product description for a B2B SaaS analytics platform. Target audience is data engineers. Tone should be professional and technical. Focus on scalability and integration capabilities. Here's an example of our style...**

You can use techniques like few-shot learning where you provide examples of inputs and desired outputs. You can use chain-of-thought prompting where you ask the model to think step-by-step. You can use role-based prompting: You are an expert data scientist with 10 years of experience.

The advantage of prompt engineering is that it's immediate, free, and requires no infrastructure. The limitation is that you can't teach the model new facts it doesn't know. If information isn't in its training data, no amount of clever prompting will help.

Strategy Two: RAG - Retrieval Augmented Generation

This is what we'll focus on today in detail. RAG solves the knowledge problem by giving the LLM access to external information at query time. You're essentially providing the model with a reference textbook it can consult before answering.

RAG is perfect when you need the model to work with proprietary documents, up-to-date information, or domain-specific knowledge that wasn't in the training data. It's dynamic - you can update your knowledge base without touching the model. And it's transparent - you can see exactly what information the model used to generate its answer.

The trade-off is added complexity. You need vector databases, embedding models, retrieval systems. There's also latency because you're doing a search before every generation. But for most business applications requiring factual accuracy, RAG is the sweet spot.

Strategy Three: Fine-tuning

Fine-tuning means taking a pre-trained model and training it further on your specific data. This actually modifies the model's weights, teaching it new patterns, writing styles, or specialized knowledge.

Fine-tuning is excellent when you need the model to adopt a very specific tone, format, or reasoning style. For example, if you need all outputs to follow a particular company writing style, or if you're doing specialized tasks like medical diagnosis or legal document analysis.

The downsides are significant. Fine-tuning is expensive - you need quality training data, compute resources, and expertise. It's static - the model's knowledge is frozen at training time. If facts change, you need to fine-tune again. And there's a risk of catastrophic forgetting where the model loses its general capabilities.

For most applications, fine-tuning is overkill. Use it when you've exhausted RAG and prompt engineering and still need better performance on specific tasks.

Strategy Four: Model Selection and Configuration

Sometimes the solution is simply using a better or more appropriate model. GPT-4 is more accurate than GPT-3.5. Claude Sonnet might be better at certain reasoning tasks. Open-source models like Llama might work fine for simpler tasks at lower cost.

You can also tune parameters. Temperature controls randomness - set it to 0 for deterministic, factual outputs, or higher for creative tasks. Top-p sampling controls diversity. Max tokens limits response length.

Consider using specialized models for specific tasks. Embedding models for similarity search, smaller models for classification, larger models for complex reasoning.

Strategy Five: Hybrid Approaches and Orchestration

In production, you often combine multiple strategies. You might use RAG to retrieve relevant context, apply prompt engineering to structure how that context is used, configure the model appropriately for your task, and even fine-tune for specific formatting needs.

You might also use multiple models in a pipeline. A smaller model for initial filtering, a larger model for the final generation. Or use the LLM to generate structured data, then validate it with deterministic code.

Key Insight

Improving LLM accuracy isn't about finding one magic solution. It's about understanding your specific requirements - what accuracy do you need, what's your budget, how dynamic is your data, what latency can you tolerate - and choosing the right combination of techniques.

For most business applications requiring factual accuracy with proprietary data, RAG combined with good prompt engineering is your best starting point.

Section 2: Prerequisites for RAG

Duration: 4 minutes

Before you can build a RAG system, you need several components in place. Let me walk through each one so you understand what you're getting into.

Prerequisite One: Your Knowledge Base

This is the most obvious but often underestimated requirement. You need documents, data, or information that you want the LLM to access. This could be PDFs, text files, web pages, documentation, databases, anything textual.

But here's what people miss - **your documents need to be clean and well-structured**. If your PDFs have garbled text from poor scanning, if your data has inconsistent formatting, if there are tons of irrelevant boilerplate sections, your RAG system will retrieve garbage.

Before implementing RAG, audit your knowledge base. Can you access these documents programmatically? Are they in readable formats? Do they contain the information users actually need? You might need to do preprocessing - extracting text from PDFs, cleaning HTML, standardizing formats.

Prerequisite Two: An Embedding Model

Embeddings are the mathematical foundation of RAG. An embedding model converts text into vectors - lists of numbers that capture semantic meaning. Similar pieces of text get similar vectors.

You need to choose an embedding model. Popular options include OpenAI's text-embedding models, open-source models like sentence-transformers, or specialized models like Cohere's embeddings.

Key considerations: What's the dimension size? Higher dimensions can capture more nuance but cost more to store and search. What's the context window? This limits how much text you can embed at once. What languages does it support? Is it optimized for your domain?

For most applications, OpenAI's text-embedding-3-small or an open-source sentence-transformer model work perfectly fine. Don't overthink this initially.

Prerequisite Three: A Vector Database

Once you have embeddings, you need somewhere to store them that enables fast similarity search. Regular databases aren't designed for this. Vector databases are specialized for storing embeddings and finding similar vectors quickly.

Options include managed services like Pinecone, open-source solutions like Weaviate or Qdrant, or lightweight options like Chroma or FAISS for prototyping.

You need to understand the trade-offs. Managed services are easier but cost more and require data to leave your infrastructure. Self-hosted gives you control but requires maintenance. Some vector DBs are better at scale, others are easier to get started with.

For learning and prototyping, I recommend Chroma - it's free, runs locally, requires minimal setup. For production, evaluate based on your scale and infrastructure.

Prerequisite Four: An LLM API

You need access to a language model for the generation phase. This could be OpenAI's API, Anthropic's Claude, open-source models you host yourself, or models through platforms like Azure or AWS.

Budget matters here. OpenAI charges per token. Self-hosting requires GPU infrastructure. Consider your expected query volume and calculate costs accordingly.

Also consider the model's context window - how much retrieved text can you stuff into the prompt alongside the user's question? GPT-4 Turbo has a 128k token context window. Smaller models might have only 4k or 8k. This affects how much retrieved context you can provide.

Prerequisite Five: Technical Infrastructure

You need basic development capabilities. Python is the standard language because all the libraries are there - LangChain, LlamaIndex, the vector database clients, embedding libraries.

You'll need API keys for your LLM provider and possibly your embedding provider. You'll need somewhere to run your code - could be your laptop for prototyping, or cloud infrastructure for production.

If you're building something user-facing, you'll need a web framework. Streamlit is great for quick demos, FastAPI for production APIs, and you might need a frontend framework for the user interface.

Prerequisite Six: Evaluation Strategy

This is often forgotten but critical. Before you build RAG, decide how you'll measure if it's working. You need test questions with known good answers. You need to define what success means - is it factual accuracy, response time, user satisfaction?

Create an evaluation dataset early. Maybe 20 to 50 representative questions that cover different types of queries your system will handle. You'll use these to iterate and improve your RAG system.

The good news is that you don't need all of this perfectly in place to start. You can begin with documents in a folder, Chroma as your vector database, OpenAI for both embeddings and generation, and a Python script. But understanding these prerequisites helps you plan for production.

Section 3: What is RAG?

Duration: 3 minutes

RAG stands for Retrieval Augmented Generation, and let me break down why each word matters.

Retrieval means we're searching for and fetching relevant information from a knowledge base. This isn't just keyword search like Google. It's semantic search - finding documents that mean the same thing as the query even if they use different words.

Augmented means we're enhancing or enriching something. Specifically, we're augmenting the LLM's prompt with additional context. The LLM doesn't natively know about your data, so we're giving it that knowledge at query time.

Generation is what LLMs do best - generating text based on input. But now that input includes both the user's question and the relevant retrieved context.

> *Here's the core insight of RAG: LLMs are excellent at reasoning, synthesis, and language generation, but they only know what was in their training data. RAG lets us keep the LLM's capabilities while giving it access to any information we want.*

Think of it like an open-book exam versus a closed-book exam. In a closed-book exam, you only have what you've memorized. That's a standard LLM. In an open-book exam, you can consult reference materials. That's RAG. The student's intelligence hasn't changed, but they can now give much more accurate, detailed answers because they have references to consult.

The beauty of RAG is that it's dynamic. Add new documents to your knowledge base, and the system can immediately use them. Update existing documents, and the next query will use the updated information. No retraining, no fine-tuning, just updated context.

RAG also provides transparency. You can show users exactly which documents or passages were used to generate the answer. This builds trust and allows for verification. In high-stakes domains like medicine or law, this auditability is crucial.

The typical use cases for RAG include: customer support systems that need to reference product documentation, internal knowledge bases where employees query company policies and procedures, research assistants that work with large document collections, question-answering systems for technical documentation, and any application where the LLM needs domain-specific or proprietary knowledge.

Section 4: Basic RAG Workflow in Detail

Duration: 9 minutes

RAG has two distinct phases: the indexing phase where we prepare our knowledge base, and the retrieval phase where we answer user queries. Let me walk through both in detail with actual code.

Phase One: Indexing

This happens once upfront, or whenever you update your documents.

Step One: Loading Documents

```
from langchain.document_loaders import DirectoryLoader, PyPDFLoader

loader = DirectoryLoader(
    './company_policies/',
    glob='**/*.pdf',
    loader_cls=PyPDFLoader
)
documents = loader.load()
print(f'Loaded {len(documents)} documents')
```

Step Two: Splitting into Chunks

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    length_function=len,
)

chunks = text_splitter.split_documents(documents)
print(f'Created {len(chunks)} chunks')
```

Chunk size of 1000 characters is roughly 200-250 tokens. Chunk overlap of 200 characters ensures information spanning boundaries appears complete in at least one chunk.

Step Three: Generating Embeddings

```
from langchain.embeddings import OpenAIEmbeddings
```

```
embeddings = OpenAIEmbeddings(
    model='text-embedding-3-small'
)
```

Step Four: Storing in Vector Database

```
from langchain.vectorstores import Chroma

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory='./chroma_db'
)

print('Vector database created and persisted')
```

Phase Two: Retrieval and Generation

Step One: User Query

User asks: What is the remote work policy?

Step Two: Embed the Query

```
query = 'What is the remote work policy?'
query_vector = embeddings.embed_query(query)
```

Step Three: Similarity Search

```
retrieved_docs = vectorstore.similarity_search(
    query=query,
    k=4
)

for i, doc in enumerate(retrieved_docs):
    print(f'Chunk {i+1}:')
    print(doc.page_content[:200])
```

Step Four: Construct Prompt

```
from langchain.chat_models import ChatOpenAI
from langchain.prompts import ChatPromptTemplate

prompt_template = ChatPromptTemplate.from_template('''
Answer based on the following context.
If the answer is not in the context, say so.

Context: {context}

Question: {question}

Answer:
''')

context = '\n\n'.join([doc.page_content for doc in retrieved_docs])
formatted_prompt = prompt_template.format(
    context=context,
    question=query
)
```

Step Five: Generate Answer

```
llm = ChatOpenAI(
    model='gpt-3.5-turbo',
    temperature=0
)

response = llm.invoke(formatted_prompt)
answer = response.content
print('Answer:', answer)
```

Complete RAG System

```
from langchain.document_loaders import DirectoryLoader, TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA

# INDEXING PHASE
loader = DirectoryLoader('./docs/', glob='**/*.txt')
documents = loader.load()

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = text_splitter.split_documents(documents)

embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(
    chunks,
    embeddings,
    persist_directory='./db'
)

# RETRIEVAL PHASE
llm = ChatOpenAI(temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectorstore.as_retriever(search_kwargs={'k': 4}),
    return_source_documents=True
)

query = 'What is the vacation policy?'
result = qa_chain.invoke({'query': query})

print('Answer:', result['result'])
print('Sources:', [doc.metadata['source'] for doc in result['source_documents']])
```

Behind the Scenes Example

User asks: **How many vacation days do I get?**

The query converts to a vector. A relevant chunk might be: Full-time employees receive 15 vacation days per year, accrued monthly. This chunk also has a vector.

Similarity search scores chunks: vacation policy chunk scores 0.89, sick leave 0.34, parking 0.12. Top 4 chunks are retrieved.

The LLM receives the context and generates: Full-time employees receive 15 vacation days per year, which accrue monthly.

The answer is grounded in actual documents, not hallucinated from training data.

Section 5: Best Practices

Duration: 2 minutes

Chunk size matters. Too small loses context. Too large retrieves noise. Start with 1000 characters.

Number of chunks is a trade-off. More context but more noise. Start with 3-5 chunks.

Embeddings quality is foundational. Poor embeddings mean poor retrieval regardless of other optimizations.

Monitor retrieval separately. Log what gets retrieved and iterate on chunking if needed.

Handle retrieval failures. Instruct the LLM to admit when answers aren't in the context.

Manage costs. Every query requires embedding, search, and LLM generation. Consider caching and smaller models.

Important Note

RAG is not a magic bullet. It works best for factual retrieval. Not ideal for creative tasks or when models need to learn patterns - that's fine-tuning territory.

Closing

Duration: 1 minute

Let's recap. We covered different approaches to improving LLM accuracy. RAG is the sweet spot for most business applications with proprietary data.

We covered prerequisites: clean documents, embedding model, vector database, LLM API, and evaluation strategy.

We did a deep dive into RAG - indexing phase and retrieval phase with complete code examples.

> *The key insight: RAG transforms LLMs from impressive but limited models into systems that accurately answer questions about your specific domain and up-to-date information.*

I'm happy to take questions about implementation details, choosing tools, or applying RAG to specific use cases. Thank you.

RAG Fundamentals & LLM Accuracy - Complete Session Script

Duration: 20-25 minutes

Complete script with code examples and delivery tips